

Работа с сервисными событиями в рамках одного рабочего процесса

Частным случаем [работы с событиями](#) является ситуация, когда источник и подписчик события совпадают, а Брокер не участвует в обмене сообщения события. Механизм подписки позволяет в пределах одного рабочего процесса ОС публиковать и обрабатывать одно и то же событие, когда оно не выходит за пределы этого процесса. Часто применяется для реализации [принципа инверсии зависимостей](#), например, для расширения стандартной логики поведения при подключении дополнительных сервисных модулей.

Например, при удалении документа нужно выполнять какую-то логику в зависимости от типа документа:

1. В методе, удаляющем документ, публикуем событие методом LocalPublish().
2. Если обработчиков у события нет, то ничего не выполняется.
3. Если в каком-то модуле сервиса нужно выполнить специфичный для сервиса код, то вызываем синхронный обработчик методом SetLocalCallback().
4. Если требуется выполнять ресурсозатратные расчеты, то вызываем асинхронный обработчик события методом SetAsyncCallback().

Публикация события

Для публикации события из C++ используйте метод LocalPublish(). Все синхронные обработчики этого события должны возвращать объект типа Result, иначе будет создано исключение при сериализации. Если результат работы обработчиков не важен, то в параметрах шаблона необходимо указать void.

Для публикации события из Python используют одноимённый метод [LocalPublish\(\)](#), когда результат выполнения обработчика не важен. При публикации события LocalPublish() всегда будет возвращать None, а не любой результат.

Метод	Аргументы	Описание
LocalPublish() Используется для синхронного вызова обработчика события в том же рабочем процессе сервиса. Синхронные обработчики вызываются сразу после публикации события.	1. channel	Имя канала, оно же идентификатор события. Используется для подключения слушателя к конкретному событию. Имя должно соответствовать установленным правилам именования событий .
	2. message	Так называемый payload, то есть объект события, приходит в теле сообщения. В качестве payload можно использовать объект любого платформенного типа, сериализуемого в json. Например, string, int, double, DateTime, Record, RecordSet, HashTable. Этот аргумент является необязательным и имеет значение по умолчанию, сериализующееся в json в null.
	3. ignore_errors	Необязательный аргумент, имеющий тип bool и значение по умолчанию false. Если установлен в значение true, при вызове обработчиков события подавляются и игнорируются кидаемые обработчиками исключения. В результирующий вектор будут помещены объекты типа Result,

		полученные от обработчиков, не бросивших исключение.
	4. parallel_execution	Необязательный аргумент, имеющий тип bool и значение по умолчанию false. Если установлен в значение true, обработчики события вызываются в отдельных потоках, обеспечивая таким образом параллельный вызов обработчиков. Важно: обработчики в этом случае вызываются в отдельных транзакциях и не видят изменения, произведенные в БД другими обработчиками или основным методом.

В остальных случаях в Python для получения результата выполнения обработчика необходимо использовать одну из функций в зависимости от типа возвращаемого значения:

- [LocalPublishBool\(\)](#)
- [LocalPublishInt\(\)](#)
- [LocalPublishDouble\(\)](#)
- [LocalPublishString\(\)](#)
- [LocalPublishDate\(\)](#)
- [LocalPublishTime\(\)](#)
- [LocalPublishDateTime\(\)](#)
- [LocalPublishMoney\(\)](#)
- [LocalPublishUuid\(\)](#)
- [LocalPublishRpcFile\(\)](#)
- [LocalPublishHashTable\(\)](#)
- [LocalPublishRecord\(\)](#)
- [LocalPublishRecordSet\(\)](#)
- [LocalPublishArrayBool\(\)](#)
- [LocalPublishArrayInt\(\)](#)
- [LocalPublishArrayDouble\(\)](#)
- [LocalPublishArrayString\(\)](#)
- [LocalPublishArrayDate\(\)](#)
- [LocalPublishArrayTime\(\)](#)
- [LocalPublishArrayDateTime\(\)](#)
- [LocalPublishMoney\(\)](#)
- [LocalPublishArrayUuid\(\)](#)

Методы возвращают list с элементами указанного типа.

Ниже показаны примеры публикации событий для рабочего процесса сервиса:

- C++

```

1  #include <sbis-event-subscription/publish.hpp>
2  Record message;
3
4  // Публикуем событие в рабочем процессе.
5  // В параметре шаблона указали возвращаемый результат – Int64.
6  std::vector<Int64> result = event::LocalPublish< Int64 >(
7      L"inside.newshasbeenpublished", message );
8  Int64 sum = 0;
9  std::for_each( result.begin(), result.end(), [&sum](Int64 val) { sum

```

```
+= val } );
```

- Python

```
1 import event
2 message = Record()
3 result = event.LocalPublishInt("auth.passwordhasbeenforgotten",
  message);
4 sum = 0
5 for n in result:
6     sum += n
```

При каких настройках будет работать публикация событий

Для C++

1. В исходном cpp-файле подключите заголовочный файл sbis-event-subscription/publish.hpp.
2. В свойствах сервисного модуля, в котором расположен cpp-файл, добавьте зависимость от сервисного модуля EventSubscription. Последний должен быть включен в состав сервиса приложения.
3. В пользовательской библиотеке в файле CMakeLists.txt добавьте sbis_add_sdk_dependency("sbis-event-subscription300").

Для Python

1. В исходном py-файле импортируйте модуль event.
2. В свойствах сервисного модуля, в котором расположен py-файл, добавьте зависимость от сервисных модулей EventSubscription и EventSubscription-Py. Последние должны быть включены в состав сервиса приложения.

Подписка на событие

Способы создания подписки из исходного кода C++ и Python различаются. Во-первых, вам нужно определить функцию/метод, в теле которой будет создана подписка на событие.

- Для C++ создайте функцию OnEndAllLoadModules в любом исходном файле, который принадлежит сервисному модулю.
- Для Python создайте метод OnEndAllLoadModules внутри файла on_event.py. Этот файл должен находиться в сервисном модуле. Если он отсутствует, создайте новый файл самостоятельно.

Затем в теле функции/метода создайте подписку на сервисные события с помощью методов SetLocalCallback() или SetAsyncCallback(), сигнатура которых описана в таблице ниже. Если в разных сервисных модулях приложения созданы подписки на одно и то же сервисное событие, то обработчики вызываются в порядке инициализации таких модулей.

SetLocalCallback

Используется для синхронного вызова обработчика события в том же рабочем процессе сервиса. Синхронные обработчики вызываются сразу после публикации события.

Аргументы	Описание
1. channel	Имя канала, оно же идентификатор события. Имя должно соответствовать установленным правилам именования событий .
2. method_name	Обработчик — имя метода БЛ . При получении указанного события будет вызываться данный метод бизнес-логики. В качестве его параметров будет передано то, что указано в качестве последующих аргументов (дополнительные аргументы обработчика). Если среди них где-то используется маркер event::MsgBody_, то на его место будет выполнена подстановка payload события.

Дополнительные аргументы обработчика.

Для указания объекта события (payload) используется маркер `event::MsgBody_` (в C++) или `event.MsgBody_` (в Python). `_MsgBody_` — экземпляр класса `Placeholder`, созданный для единообразия с C++. Вместо `event::MsgBody_` можно использовать класс [sbis::rpc::Placeholder](#) (в C++) или [sbis.Placeholder](#) (в Python).

SetAsyncCallback

Данный метод используется для асинхронного вызова обработчика события на той же группе сервисов под пользователем, который опубликовал событие.

Метод БЛ, вызываемый в качестве обработчика события, должен быть включен в состав [публичного API](#).

Внимание: `SetAsyncCallback()` полностью подчиняется правилам [асинхронных вызовов](#).

Аргументы	Описание
1. channel	Имя канала, оно же идентификатор события.
2. method_name	Обработчик — имя метода БЛ . При получении указанного события будет вызываться данный метод бизнес-логики. В качестве его параметров будет передано то, что указано в качестве последующих аргументов (дополнительные аргументы обработчика). Если среди них где-то используется маркер <code>event::MsgBody_</code> , то на его место будет выполнена подстановка payload события.
3. Политика публикации события (опционально)	Объект типа <code>publication policy</code> . Существует три политики: • <code>ppIMMEDIATELY</code> — событие отсылается сразу. • <code>ppON_TRANSACTION_COMMIT</code> — событие отсылается при успешном завершении текущей транзакции. Если транзакция отсутствует, то публикуется сразу. • <code>ppON_REQUEST_FINISH</code> (значение по умолчанию) — событие отсылается при завершении обработки входящего запроса самого первого метода, пришедшего на сервер.
Дополнительные аргументы обработчика.	Для указания объекта события (payload) используется маркер <code>event::MsgBody_</code> (в C++) или <code>event.MsgBody_</code> (в Python). <code>_MsgBody_</code> — экземпляр класса <code>Placeholder</code> , созданный для единообразия с C++. Вместо <code>event::MsgBody_</code> можно использовать класс sbis::rpc::Placeholder (в C++) или sbis.Placeholder (в Python).

В основном методе в транзакции вносятся изменения в базу и публикуется событие для информирования об этих изменениях. При этом в методе есть какие-то действия после транзакции, которые могут привести к завершению метода с ошибкой.

```
1 with CreateTransaction(TransactionLevel.READ_COMMITTED,  
2   TransactionMode.WRITE):  
3     # Какая-то логика внутри транзакции.  
4     sbis.SqlQuery("UPDATE table SET field = $1 WHERE id = $2",  
5       new_value, id)  
6     event.LocalPublish("table.changed", id)  
7     # Какая-то логика вне транзакции.  
8     if something_wrong:  
9       raise sbis.Error("Ошибка при выполнении такого-то действия")
```

Если навесить "тяжелый" асинхронный обработчик, то, в случае возникновения исключения в основном методе, обработчик вызван не будет. Изменения в БД будут внесены, но постобработка для них выполнена не будет.

```
1 event.SetAsyncCallback("table.changed", "Table.OnChangeLongHandler",  
2   event.MsgBody_ );
```

Правильнее навесить "легкий" синхронный обработчик события, в котором, при успешном коммите транзакции, вызвать асинхронно "тяжелый" обработчик.

```
1 event.SetLocalCallback("table.changed", "Table.OnChange",
2   event._MsgBody_ );
3 def Table.OnChange(event_body):
4     def on_commit():
5         sbis.BLObject('Table').AsyncInvoke('OnChangeLongHandler',
6   event_body)
7     sbis.InvokeOnTransactionCommit(on_commit)
```

Примеры

- C++

```
1 #include <sbis-event-subscription/set_local_callback.hpp>
2 #include <sbis-event-subscription/set_async_callback.hpp>
3 extern "C" MODULE_DLLFUNC bool OnEndAllLoadModules(
4   sbis::EndLoadEventAnswer* ) {
5     // Метод "Объект.СинхронныйОбработчик" в данном примере должен
6     // возвращать Int64, т.к. этот тип указан при публикации события.
7     event::SetLocalCallback( L"inside.newshasbeenpublished",
8   L"Новости.ОтправитьУведомлениеСотрудникам", event::_MsgBody_, params
9   );
10    // Метод "Новости.ОтправитьУведомлениеСотрудникам" может
11    // возвращать результат любого типа, т.к. он вызывается асинхронно.
12    // В данном случае метод будет вызван (асинхронно) в точке
13    // завершения текущего RPC-вызова и только при условии его успешного
14    // завершения
15    event::SetAsyncCallback( L"inside.newshasbeenpublished",
16   L"Новости.ОтправитьУведомлениеСотрудникам", params );
17    // Метод "Новости.ОтправитьУведомлениеСотрудникам" может
18    // возвращать результат любого типа, т.к. он вызывается асинхронно.
19    // В данном случае метод будет вызван (асинхронно) в точке
20    // публикации события, а не по (успешному) завершению RPC-вызова
21    event::SetAsyncCallback( L"inside.newshasbeenpublished",
22   L"Новости.ОтправитьУведомлениеСотрудникам",
23   PublicationPolicy::ppIMMEDIATELY params );
```

- Python

```
1 def OnEndAllLoadModules(answer):
2     import event
3     event.SetLocalCallback("auth.passwordhasbeenforgotten",
4   "Пользователь.ВосстановитьПарольПользователя", event._MsgBody_,
5   params)
6     # Метод "Пользователь.ВосстановитьПарольПользователя" может
7     // возвращать результат любого типа, т.к. он вызывается асинхронно.
8     # В данном случае метод будет вызван (асинхронно) в точке
9     // завершения текущего RPC-вызова и только при условии его успешного
10    // завершения
11    event.SetAsyncCallback("auth.passwordhasbeenforgotten",
12   "Пользователь.ВосстановитьПарольПользователя", params )
```

```
7
8     # Метод "Пользователь.ВосстановитьПарольПользователя" может
возвращать результат любого типа, т.к. он вызывается асинхронно.
9     # В данном случае метод будет вызван (асинхронно) в точке
публикации события, а не по (успешному) завершению RPC-вызова
10    event.SetAsyncCallback("auth.passwordhasbeenforgotten",
"Пользователь.ВосстановитьПарольПользователя",
PublicationPolicy.ppIMMEDIATELY, params )
```

Отписка от события

При работе с событиями в рамках одного рабочего процесса производить отписку от события не требуется.